



Day 14 Notes: Transformer Decoder

TRANSFORMER DECODER

Complete Visual Documentation

From Encoder Output → Decoder Architecture → Token Generation

Encoder Output

[seq x 512]
Context Matrix

Masked Self-Attn

Causal Mask
No Future Peek

Cross-Attention

Q=Decoder
K,V=Encoder

Feed-Forward

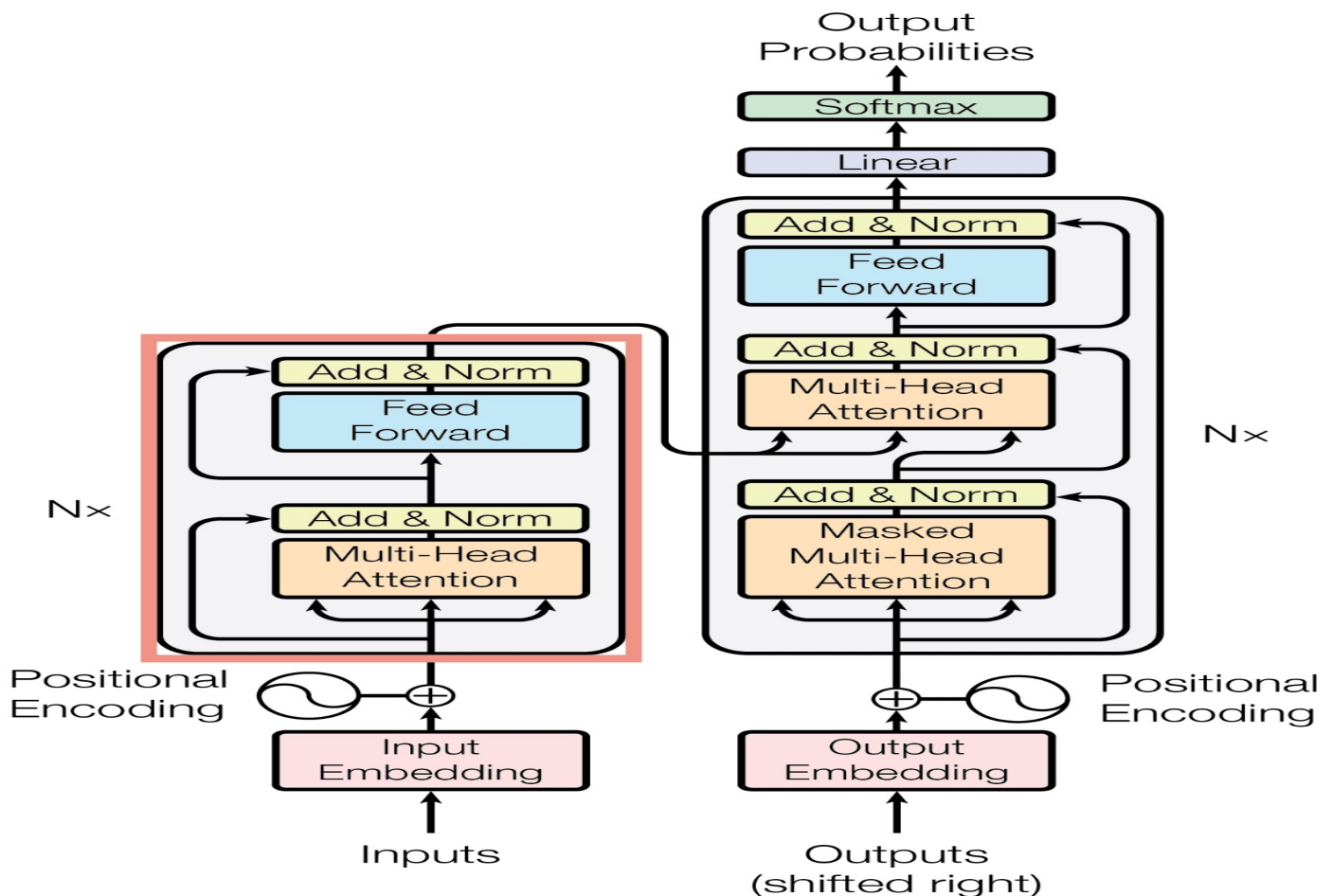
Linear→ReLU
→Linear

Add & Norm

Residual +
Normalise

TABLE OF CONTENTS

- 01 Encoder Output — What exactly comes out?
- 02 Decoder Input — Autoregressive generation explained
- 03 Component 1 — Masked Multi-Head Self-Attention
- 04 Component 2 — Cross-Attention (Encoder-Decoder)
- 05 Component 3 — Position-wise Feed-Forward Network
- 06 Component 4 — Add & Layer Normalisation
- 07 Full Decoder Architecture Diagram
- 08 Quick Reference & Summary Tables



What is the Encoder Output?

The Encoder processes the entire input sentence and produces a rich, context-aware numerical matrix for every token. This is called the ENCODER MEMORY or ENCODER OUTPUT. It is passed to every Decoder layer as Key (K) and Value (V) in Cross-Attention.

ANALOGY

Think of the Encoder as a super-smart reader.
You give it: "I love AI"
It reads every word, understands relationships, and produces a detailed 'summary card' for EACH word — packed with context.
That collection of summary cards = Encoder Output.

Shape of the Encoder Output

For input "I love AI" (3 tokens) with `d_model = 512`:

Token	Position	Embedding Vector	What it encodes
"I"	pos 1	[0.21, -0.45, 0.88, ... 512 nums]	Subject, singular pronoun
"love"	pos 2	[0.93, 0.12, -0.77, ... 512 nums]	Verb, connects subj & obj
"AI"	pos 3	[-0.33, 0.67, 0.44, ... 512 nums]	Object, technology domain

Final shape: Encoder Output = [3 x 512] matrix (sequence_length x d_model)

How the Encoder builds this output — step by step

Step 1 Tokenisation	"I love AI" → ["I", "love", "AI"]
Step 2 Token Embedding	Each token mapped to 512-dim vector (learned weights)
Step 3 Positional Encoding	Sine/cosine position signals added to each vector
Step 4 6× Encoder Layers	Multi-Head Self-Attention + FFN x 6
Output Encoder Memory	Final [3 x 512] contextualised matrix → sent to Decoder

Key insight: The Encoder Output shape NEVER changes — it is always [src_len x 512].

This same matrix is fed as K, V into ALL 6 Decoder cross-attention layers.

SECTION 02 — DECODER INPUT & AUTOREGRESSIVE GENERATION

What goes INTO the Decoder?

The Decoder does NOT receive the same input as the Encoder. It receives the previously generated TARGET TOKENS, shifted one position to the right. This is called Teacher Forcing during training.

EXAMPLE: Translation Task

TASK: Translate "I love AI" (English) -> "J'aime l'IA" (French)

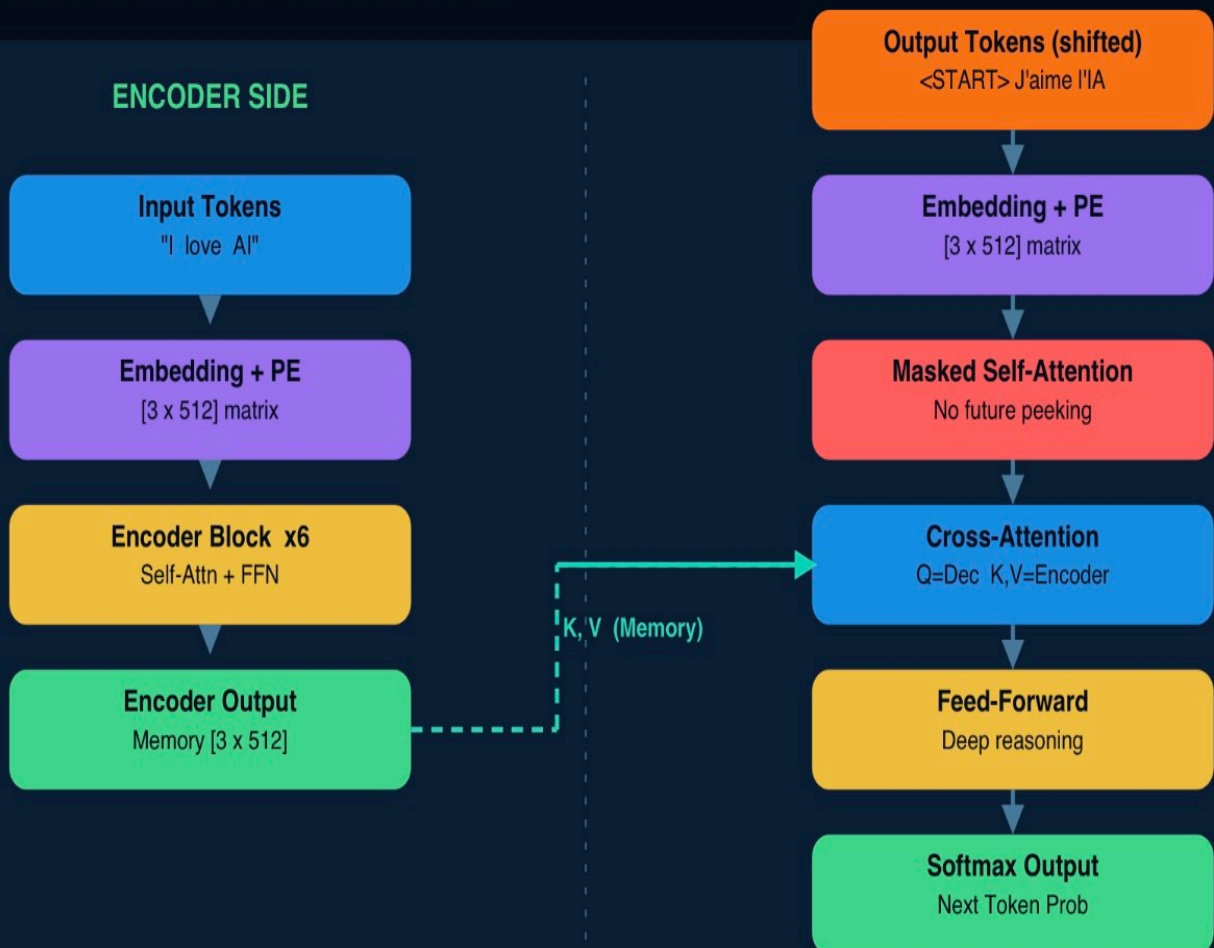
Decoder Step 1: Input = <START> -> generates "J'aime"

Decoder Step 2: Input = <START> J'aime -> generates "l'IA"

Decoder Step 3: Input = <START> J'aime l'IA -> generates <END>

This one-token-at-a-time process is called AUTOREGRESSIVE DECODING.

Complete Encoder → Decoder Data Flow



Decoder Input also uses: Token Embedding + Positional Encoding (same as Encoder input)

SECTION 03 — COMPONENT 1: MASKED MULTI-HEAD SELF-ATTENTION

What is it?

This is the FIRST sub-layer in every Decoder block. Each output token attends to all previous output tokens — but NOT future ones. A causal mask enforces this rule.

WHY THE MASK?

During TRAINING we feed the full target sentence at once (for speed).
But we must NOT allow a word to "cheat" by looking at future words.
The MASK prevents this — ensuring training matches inference behaviour.

How the Mask Works — Concrete Example

Target sequence: <START> J'aime l'IA

Token	Can Attend To	Attention Scores (after mask)	Reason
<START>	<START> only	[1.0 , 0 , 0]	Can only see itself
J'aime	<START> + J'aime	[0.4 , 0.6 , 0]	Sees past + itself
l'IA	All 3 tokens	[0.3 , 0.3 , 0.4]	Sees all past tokens

The Masking Mechanism — Technical Detail

- STEP 1: Compute raw attention scores = $QK^T / \sqrt{d_k}$
- STEP 2: Replace scores where $j > i$ (future positions) with -9999
- STEP 3: Apply softmax -> $\text{softmax}(-9999) = \sim 0.0$ (zero weight)
- STEP 4: Output = $\text{softmax}(\text{masked_scores}) \times V$

Result: each token gets context ONLY from itself and earlier tokens.

Multi-Head Attention — Why 8 Heads?

Instead of one attention pass, 8 parallel heads each learn different relationship types. Outputs are concatenated and projected back to $d_{\text{model}}=512$.

Head 1

Syntactic structure

Subject-verb relationships

Head 2

Positional proximity

Adjacent word context

Head 3

Semantic similarity

Words with related meaning

Head 4

Coreference

Pronoun to noun links

Heads 5-8

Varied patterns

Tense, gender, number etc.

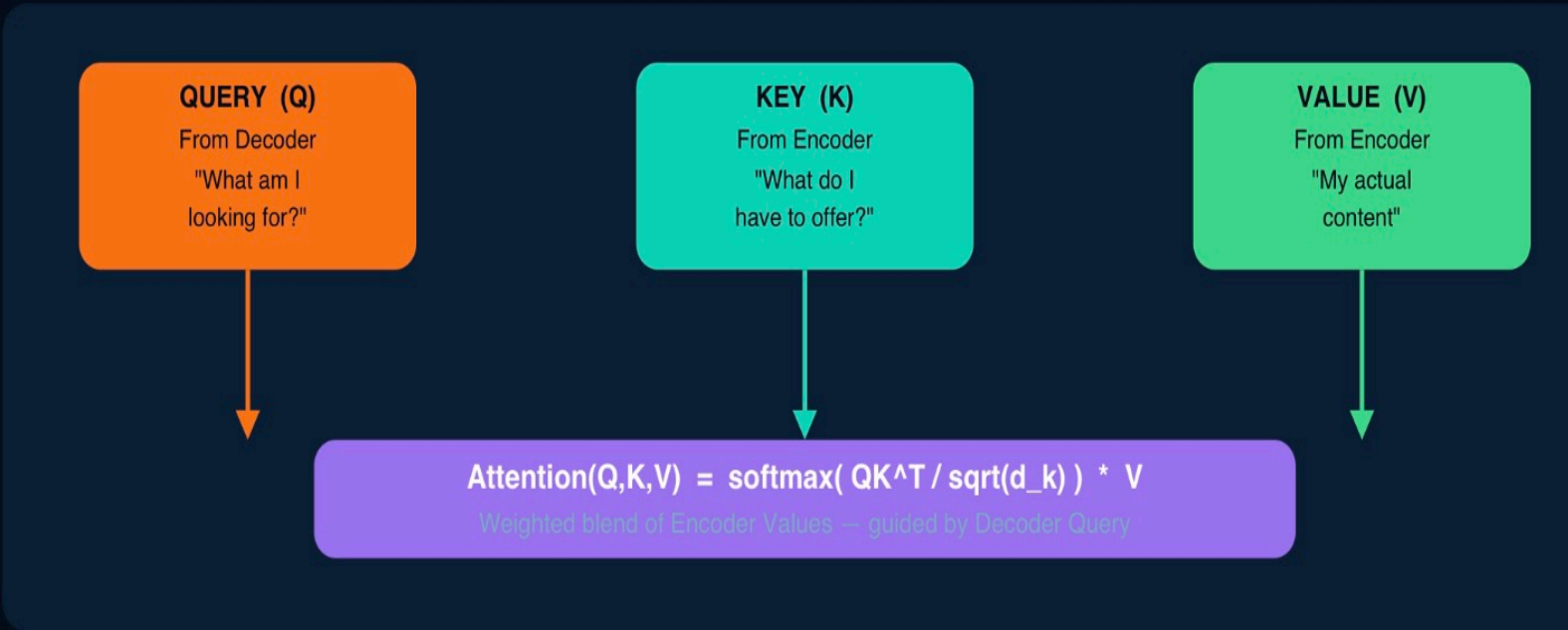
What is it?

Cross-Attention is the BRIDGE between Encoder and Decoder. It is the SECOND sub-layer and the ONLY component that directly reads the Encoder's output. Without it, the Decoder would generate output blind to the source sentence.

Q, K, V — Where Do They Come From?

Matrix	Source	Shape	Meaning
Q (Query)	Decoder prev sub-layer output	[tgt_len x 512]	"What info do I need?"
K (Key)	Encoder Output (Memory)	[src_len x 512]	"What do I have to offer?"
V (Value)	Encoder Output (Memory)	[src_len x 512]	"Here is my actual content"

Visual: Q, K, V Flow Through Cross-Attention



Concrete Translation Example

When Decoder generates "l'IA" (the French word for "AI"):

Encoder Token (Key)	Attention Score	Why?
"I"	0.05	Unrelated to "l'IA"
"love"	0.15	Verb — indirect relevance
"AI"	0.80	Direct translation target → HIGH attention

KEY INSIGHT

The Decoder token "l'IA" draws 80% of its context from the Encoder's "AI" representation. This is exactly how the model knows WHAT to translate at each step. The softmax scores form an "alignment" — mapping output tokens to input tokens.

Component 3: Position-wise Feed-Forward Network (FFN)

The FFN is the THIRD sub-layer. It is a 2-layer neural network applied INDEPENDENTLY to each token position. Positions do NOT exchange info here — that is the attention layers' job. FFN provides deeper per-position reasoning.



FORMULA: $\text{FFN}(x) = \max(0, x \cdot W1 + b1) \cdot W2 + b2$

x = input vector ($d_{\text{model}} = 512$ dimensions)

$W1$ = weight matrix 512×2048 (expand 4x)

$\text{ReLU} = \max(0, \dots)$ removes all negative values

$W2$ = weight matrix 2048×512 (compress back)

CONCRETE EXAMPLE

When Decoder generates "l'IA" after Cross-Attention has gathered context:

Input to FFN: rich vector blending "AI" from source + French grammar context

FFN reasons: "translating AI + French + definite article $\rightarrow$ output = 'l'IA'"

The FFN adds the final "thinking" step before projecting to vocabulary.

Component 4: Add & Layer Normalisation

After EVERY sub-layer (Masked Attn, Cross-Attn, FFN) there is a residual connection (Add) followed by Layer Normalisation. Both are critical for training stability in deep networks.

RESIDUAL CONNECTION (ADD)

$\text{output} = x + \text{SubLayer}(x)$

The ORIGINAL input x is added back to the sub-layer output.

This creates a "shortcut" path for gradients during backpropagation, preventing vanishing gradients in deep 6-layer stacks.

LAYER NORMALISATION

Normalises each vector to $\text{mean}=0$, $\text{std}=1$

BEFORE: $[3.2, -8.5, 0.1, 15.4]$

$\text{mean}=2.5$, $\text{std}=9.1$ (unstable!)

AFTER: $[0.4, -1.1, -0.3, 1.8]$

$\text{mean}\sim 0$, $\text{std}\sim 1$ (stable!)

Full wrapper per sub-layer:

$x \rightarrow \text{SubLayer}(x) \rightarrow \text{Add}(x, \text{SubLayer}(x)) \rightarrow \text{LayerNorm} \rightarrow \text{next sub-layer}$

Applied THREE TIMES per Decoder layer: after Masked Attn, Cross-Attn, and FFN.

One Complete Decoder Layer (repeated x6 in the original Transformer)



End-to-End Token Generation — Step by Step

#	Stage	What Happens
1	Decoder Input	Target tokens shifted right
2	Embedding + PE	Tokens → vectors + position info
3	Masked Self-Attn	Each token sees only its past
4	Add & Norm	Residual + stabilise
5	Cross-Attention	Query the Encoder memory
6	Add & Norm	Residual + stabilise
7	Feed-Forward	Deep per-position reasoning
8	Add & Norm	Residual + stabilise
9	Repeat x6 layers	Stack 6 identical Decoder blocks
10	Linear + Softmax	Project to vocabulary → probabilities
11	Argmax / Sample	Pick next token → feed back as input

SECTION 08 — QUICK REFERENCE SUMMARY

All Decoder Components at a Glance

Component	Sub-layer	Q K V Sources	Key Purpose	Unique Trait
Masked Self-Attn	1st	All from Decoder	Understand generated context	Causal mask (no future)
Cross-Attn	2nd	$Q = \text{Decoder}, K, V = \text{Encoder}$	Bridge to source sentence	Only layer using Encoder
Feed-Forward	3rd	N/A	Deep per-position reasoning	Position-wise, independent
Add & Norm	After each	N/A	Training stability	After EVERY sub-layer

Key Hyperparameters — Original Transformer (Vaswani et al. 2017)

d_model 512	Size of every embedding vector	d_ff 2048	Inner dimension of FFN (4x d_model)
h (heads) 8	Parallel attention heads	d_k 64	$d_{\text{model}} / h = 512 / 8$
N (layers) 6	Stacked Decoder (and Encoder) blocks	Vocab size ~37,000	Final linear output dimension (WMT)